

Report: Final Line Detection Pipeline

White tape detection on a dark floor using MATLAB image processing

1. Overview

The purpose of the code is to detect a white masking-tape line on a dark floor from a recorded video. The code compares different segmentation approaches, then uses the most reliable and practical method as the final pipeline. The final version uses grayscale thresholding because the tape is mainly distinguished from the floor by brightness rather than by colour. After detecting the tape, the code calculates the tape centroid and converts its horizontal position into a normalised line-position error signal.

The output figures show that grayscale and RGB-brightness segmentation detect the line almost identically. The HSV mask, using the selected thresholds $S < 0.25$ and $V > 0.70$, fails to detect the tape in the tested frames because the tape is not bright enough under the recorded lighting conditions to satisfy the value threshold. For this specific task, the important information is therefore intensity contrast, not colour information.

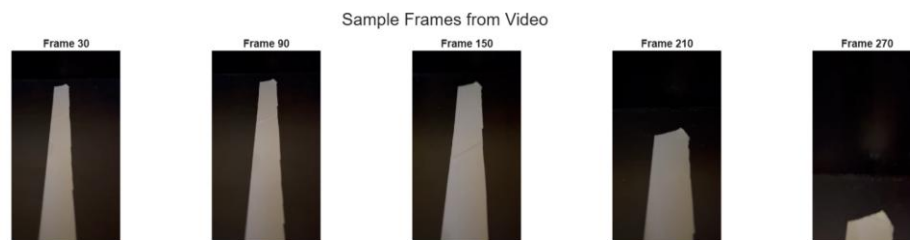


Figure 1: Sample frames extracted from the recorded video.

2. How the final code works

The final code follows the sequence below:

- **Load the video:** The script opens Video.mp4 using VideoReader, reads the frame rate, duration, number of frames and frame size.
- **Extract sample frames:** A few frames are selected from the video so that the segmentation quality can be visually checked.
- **Resize the frame:** Each frame is downscaled to 25% of the original resolution. This reduces the number of pixels processed and improves speed.
- **Convert to grayscale:** The final pipeline converts the resized RGB frame to a single grayscale image.
- **Apply a brightness threshold:** Pixels brighter than the threshold are classified as tape. In the tested code the threshold is 100 on the 0-255 grayscale scale.
- **Remove small noise blobs:** bwareaopen removes small connected components with area below the minimum area threshold.
- **Close small gaps:** Morphological closing with a disk-shaped structuring element fills small breaks and improves the continuity of the detected tape region.

- **Use the largest detected blob:** regionprops is used to find connected regions. The largest area is selected as the tape, which makes the method more robust against small bright reflections or noise.
- **Calculate the centroid:** The centroid of the selected tape region gives the detected line position in the image.
- **Calculate the error signal:** The centroid x-position is compared with the image centre to produce a normalised steering-style error signal.

The final detection is shown by overlaying the detected mask on the original sample frames. The green overlay shows that the grayscale method consistently captures the visible white tape, including when the tape appears lower in the frame near the end of the video.

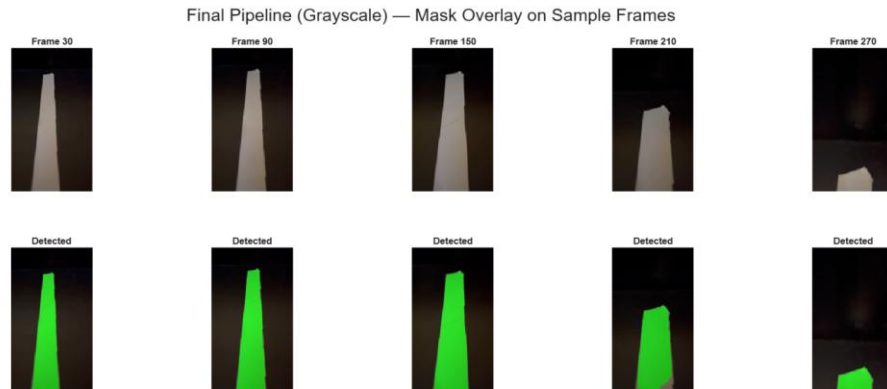


Figure 2: Final grayscale pipeline with the detected tape mask shown in green.

3. RGB versus HSV line segmentation

The code compares grayscale thresholding, RGB-brightness thresholding and HSV thresholding. The grayscale method thresholds the intensity after converting the frame to grayscale. The RGB method calculates an average brightness from the red, green and blue channels and thresholds that value. The HSV method converts the image to hue, saturation and value, then attempts to detect white tape using low saturation and high brightness.

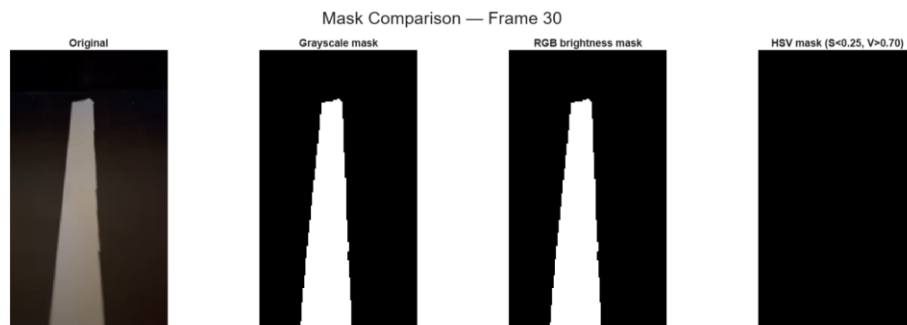


Figure 3: Visual comparison of grayscale, RGB brightness and HSV masks for Frame 30.

The visual comparison shows that grayscale and RGB-brightness segmentation produce almost the same mask. This is expected because both approaches are essentially measuring the same property: the tape is much brighter than the black floor. The HSV result is black for the tested frame, meaning that no significant tape region was detected with the selected HSV thresholds.

The quantitative comparison supports this observation. The approximate values read from the generated plots are summarised below. The exact numbers may vary slightly depending on the video frame, the resize factor and the computer used to run the timing test.

Metric	Grayscale	RGB brightness	HSV
Mean SNR over sample frames	About 8.8-9.0	About 8.5-8.8	Approximately 0
Mean mask coverage	About 18-19%	About 18%	Approximately 0%
50-frame timing at 25% resolution	1.53 ms/frame	1.30 ms/frame	1.24 ms/frame
Estimated processing speed	~655 FPS	~772 FPS	~806 FPS
Detection quality	Good	Good	Poor with chosen thresholds

The HSV method was not selected as the final method because the thresholds were too strict for this recording. White objects are normally expected to have low saturation and high value in HSV, but the recorded tape appears dimmer because of uneven lighting, distance from the camera and the dark scene. As a result, many tape pixels do not pass the $V > 0.70$ condition. The HSV approach could be improved by lowering the value threshold or by using adaptive thresholds, but this would add extra tuning without providing a clear advantage for this video.

For this particular task, there is not a meaningful benefit in using a full colour-space method. The target line is white and the background is dark, so the separation is dominated by brightness contrast. That is why grayscale and RGB-brightness segmentation give very similar masks, while grayscale is simpler and more suitable for a final real-time pipeline.

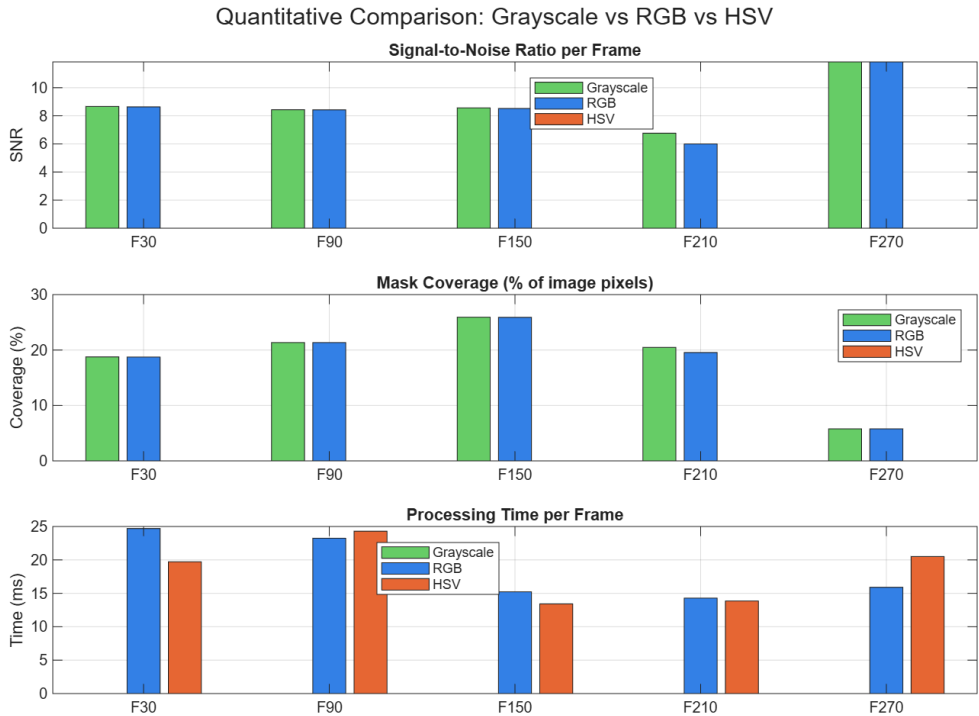


Figure 4: Quantitative comparison of signal-to-noise ratio, mask coverage and processing time.

4. Speed improvements included in the final code

The report-generation code was useful for comparing methods, producing figures and analysing the video, but that version was not the most suitable form for real-time operation. MATLAB code running frame-by-frame on a laptop is also not the same as code running on a Raspberry Pi, where processor speed and memory are more limited. For this reason, the final line-detection pipeline was simplified so that it performs only the operations needed to find the line and calculate the error signal.

The optimised real-time pipeline uses the following sequence:

- Read one video frame.
- Crop the bottom region of interest where the line is expected to appear.
- Resize the cropped region to a lower resolution.
- Convert the cropped image to grayscale.
- Apply a simple brightness threshold to detect the white tape.
- Count the detected pixels and reject frames with too few white pixels.
- Calculate the line centre using column sums.
- Calculate the normalised line-position error.

The main optimisation changes are described below.

- **Region of interest processing:** Only the lower part of each frame is processed. This is suitable for a line-following robot because the useful steering information is normally closest to the camera at the bottom of the image. Ignoring the upper part of the frame reduces the number of pixels processed and removes unnecessary background information.
- **Cropping before resizing:** The frame is cropped first and then resized. This is faster than resizing the full frame, because MATLAB only scales the part of the image that will actually be used.
- **Reduced resolution:** The region of interest is resized to 25% of its original width and height. This reduces the number of pixels to about one-sixteenth of the cropped image, which greatly reduces processing time while still preserving enough information to locate the wide white tape.
- **Grayscale instead of colour images:** The final code converts the cropped image to grayscale and works with one intensity image instead of three colour channels. This avoids RGB-channel calculations and HSV conversion. This is appropriate here because the tape is separated from the floor mainly by brightness rather than by hue or colour.
- **Simple fixed threshold:** A fixed grayscale threshold is used to classify bright pixels as tape. This is computationally cheaper than adaptive thresholding, edge detection or machine-learning methods. It is sufficient for this controlled video because the white tape has strong contrast against the dark floor.
- **Fast centroid using column sums:** The optimised version avoids heavier connected-component operations for the real-time step. Instead, it sums the binary mask by column and calculates the weighted average x-position of the detected white pixels. This gives the line centre directly with very little computation.
- **Limited display and plotting:** Displaying every frame and continuously updating figures can slow down MATLAB. The real-time version allows the display to be switched off, or only updated every few frames. The final analysis plots are generated after processing rather than during every frame.

The speed comparison in the earlier analysis showed that grayscale, RGB brightness and HSV processing were all fast at 25% resolution. However, the final choice was not based only on timing. HSV did not produce a useful mask with the selected thresholds, while grayscale provided reliable detection with the simplest

implementation. Therefore grayscale was selected as the best method for the final resource-constrained pipeline.

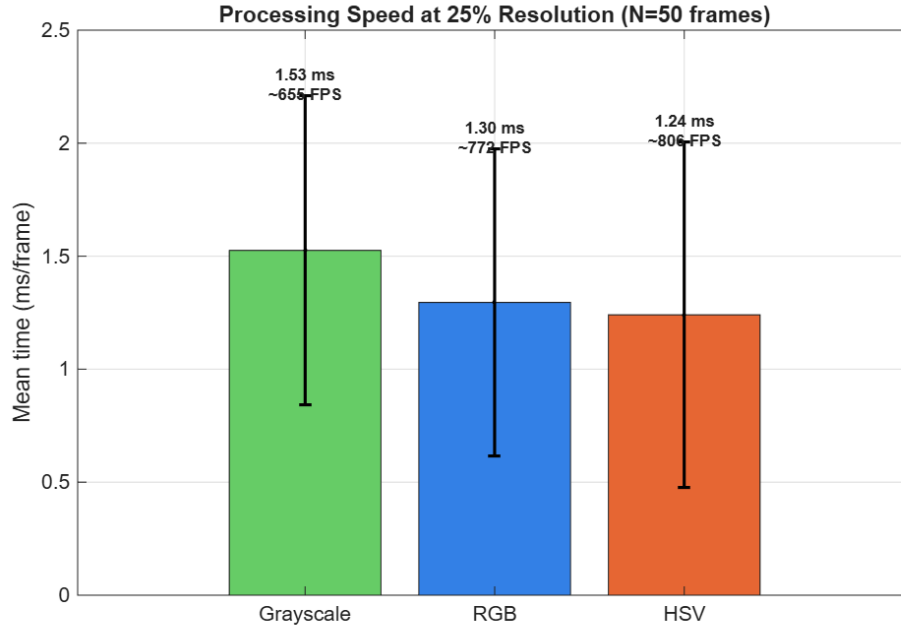


Figure 5: Initial processing speed comparison over 50 frames at 25% resolution.

After simplifying the real-time version further, the code processed the recorded video with an average processing time of approximately 3.204 ms per frame on the laptop. This corresponds to an estimated processing rate of about 312 frames per second, which is much faster than the frame rate required by a typical camera stream. The detection status plot also showed a 100% detection rate for the recorded video. Although the Raspberry Pi will be slower than the laptop, the result indicates that the algorithm is lightweight enough for real-time implementation.

In summary, the final code was made faster by reducing the processed image size, using only the useful region of interest, converting to grayscale, applying a simple threshold, avoiding unnecessary colourspace conversions and using a direct centroid calculation. These changes produce simple MATLAB code that can run very quickly while still reliably finding the white line in the recorded frames.

5. Error metric and line position over time

After the tape is segmented, the code calculates the centroid of the largest detected tape region. Only the horizontal centroid position is used for the error metric, because the aim is to determine whether the line is left or right of the image centre.

The normalised error is calculated as:

$$\text{error} = (\text{cx} - \text{image_width}/2) / (\text{image_width}/2)$$

In this equation, cx is the x-coordinate of the detected tape centroid. A value of 0 means that the tape is in the centre of the image. A negative value means the line is to the left of the centre, and a positive value means the

line is to the right. The scale is normalised so that -1 represents the far left side of the image and +1 represents the far right side.

Error value	Meaning
-1	Line is near the far left of the image
0	Line is centred
+1	Line is near the far right of the image

The plotted error signal shows that the line remains close to the image centre for most of the video. The mean error is -0.019, which indicates a slight average offset to the left. The standard deviation is 0.091, showing that the detected line position does not vary strongly during the recorded sequence. The smoothed signal follows the same trend as the raw signal but removes small frame-to-frame fluctuations, making it more suitable for later use in a control system.

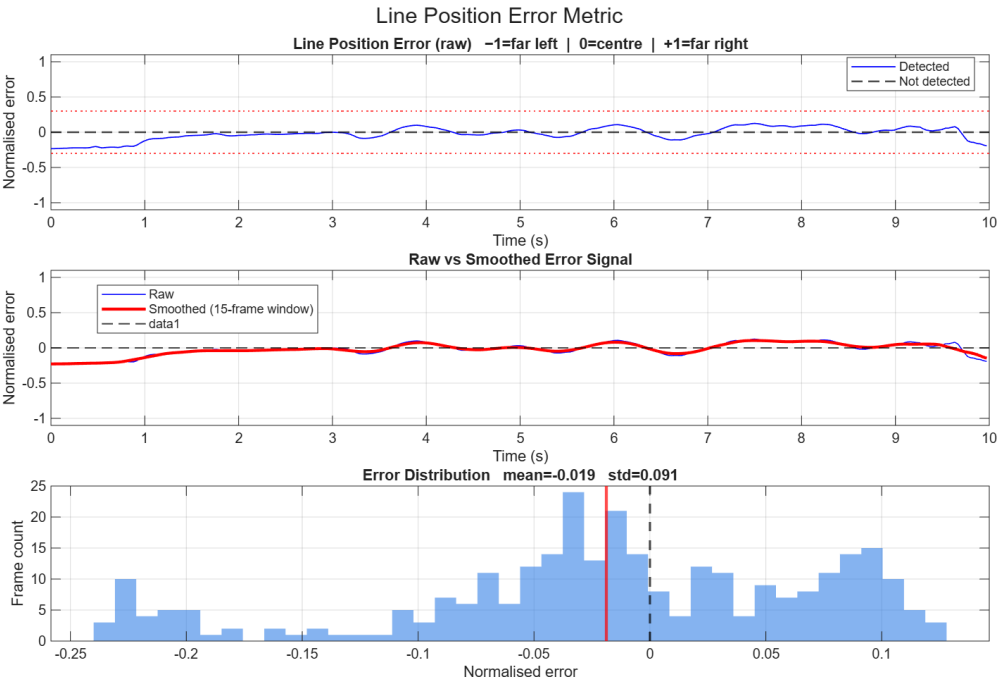


Figure 6: Raw and smoothed line-position error over time, with the error distribution.

The centroid path plot gives another view of the line movement. The dashed vertical line indicates the image centre. The coloured points show the detected centroid location over time. Most detections remain close to the centre line, confirming the result from the error plot.

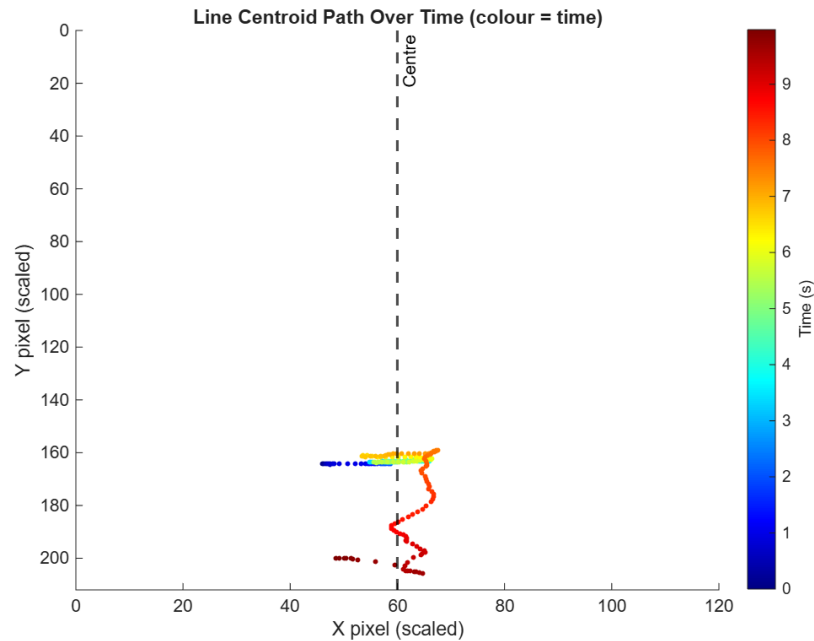


Figure 7: Detected line centroid path over time.

6. Limitations and recommended final adjustments

The current pipeline works well for this controlled video because the white tape has strong contrast against the dark floor. However, fixed thresholds can become less reliable if the lighting changes strongly, if the floor becomes reflective, or if other bright objects appear in the frame.

For a more robust final implementation, the code could include adaptive thresholding, automatic threshold calibration from the first few frames, or a dynamic region of interest. It would also be useful to test the method with different lighting conditions, different tape widths, different camera heights and robot motion.

One coding detail to check in the final MATLAB script is that timing values for grayscale, RGB and HSV should be measured in separate tic/toc blocks. This ensures that the plotted processing-time comparison is fair and that no method is accidentally assigned a zero timing value.

7. Conclusion

The best final version of the code uses a grayscale-based segmentation pipeline. This method was selected because it detects the white tape reliably, is simple to explain, and avoids unnecessary colour processing. RGB-brightness segmentation gives almost the same result because it also measures intensity contrast. HSV segmentation did not work well with the selected thresholds for this video, mainly because the tape was not bright enough in the recorded lighting to pass the high-value threshold.

The final pipeline also calculates a useful normalised error metric. For the recorded video, the mean error was -0.019 with a standard deviation of 0.091, meaning that the line stayed close to the centre with only a small left offset. This error signal is suitable as an input for a future robot steering or line-following controller.